App Intents Tutorial: A Field Guide for iOS Developers

Learn how to make your first App Intent and then go deeper into advanced topics like Apple Intelligence, Spotlight and more.

Jordan Morgan
June 26, 2024

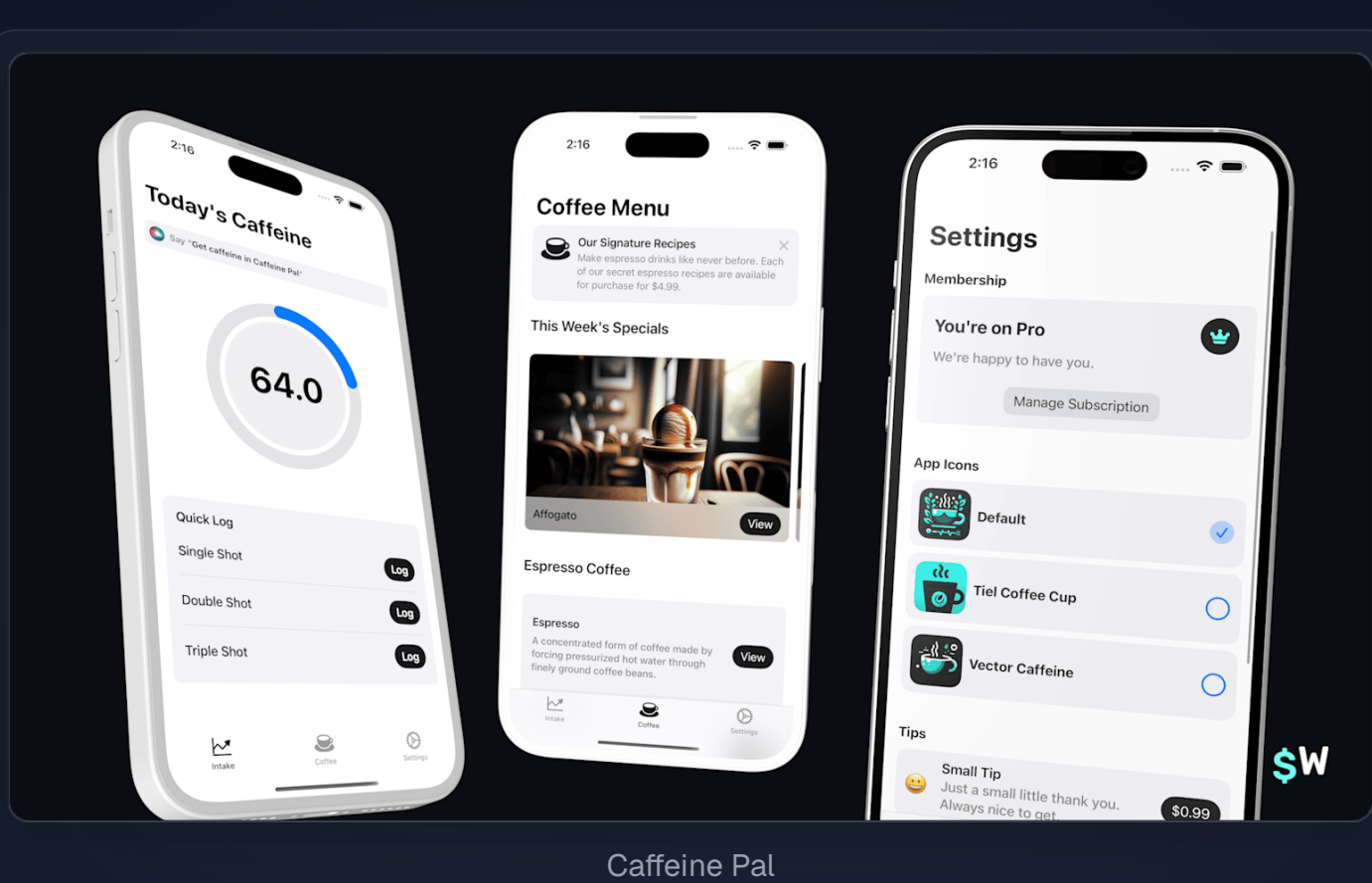
An App Intents Field Guide

Recently, Apple brought the wraps off of Apple Intelligence. A continuous theme throughout the WWDC 2024 Keynote was using the system capabilities of iOS with the personal context of your phone to let Siri do things for you. Repetitive tasks, actions you perform often, or complex workflows could all be done with Siri once Apple Intelligence rolls out.

But to do that (and more!), Siri needs to know a lot about *your* app. What can it do? What type of data does it have? What actions do people perform in it a lot?

In short, answering those questions is what bridges the gap between Siri, other system APIs, the user and your app. And, we build that bridge by using the App Intents framework. Arguably, it's the most *important* framework that developers should know about in today's landscape.

Here, we'll look at how to make an App Intent from scratch with our caffeine tracking app, Caffeine Pal. Plus, we'll see what making one simple intent can unlock for us across the system:



Caffeine Pal

When we're done, we'll see how to use our App Intent to power:

- App Shortcuts
- Spotlight search and suggestions
- Control Center and Lock Screen controls
- Interactive widgets
- Siri
- And, several other things we get for free, such as using our intents for things like the Action Button or Apple Pencil Pro squeeze gestures.

First, let's define what exactly App Intents are.

What are App Intents?

App Intents allow you to expose the things your app does to the system. Doing so allows it to be invoked in several different contexts. I know that sounds broad, but think of it this way: it allows people to do actions from your app when they aren't using it.

For each action your app can do, you can also make an intent for it. When you do that, you open up your app to the entire iOS ecosystem:

- Spotlight search and suggestions
- Shortcut actions in the Shortcuts app
- Action button
- Apple Pencil Pro
- WigetKit suggestions and Smart Stack
- Focus Filters
- Accessibility Actions
- Live Activities
- Control Center and Lock Screen controls (new in iOS 18)
- And, Apple Intelligence in iOS 18 with Assistant Schemas and an associated domain.

That list shows how App Intents are a foundational aspect of iOS. It brings your app to virtually every marquee feature of the Apple ecosystem.

So, what does an App Intent look like?

An intent itself, in its most basic form, conforms to the `AppIntent` protocol. It does three things:

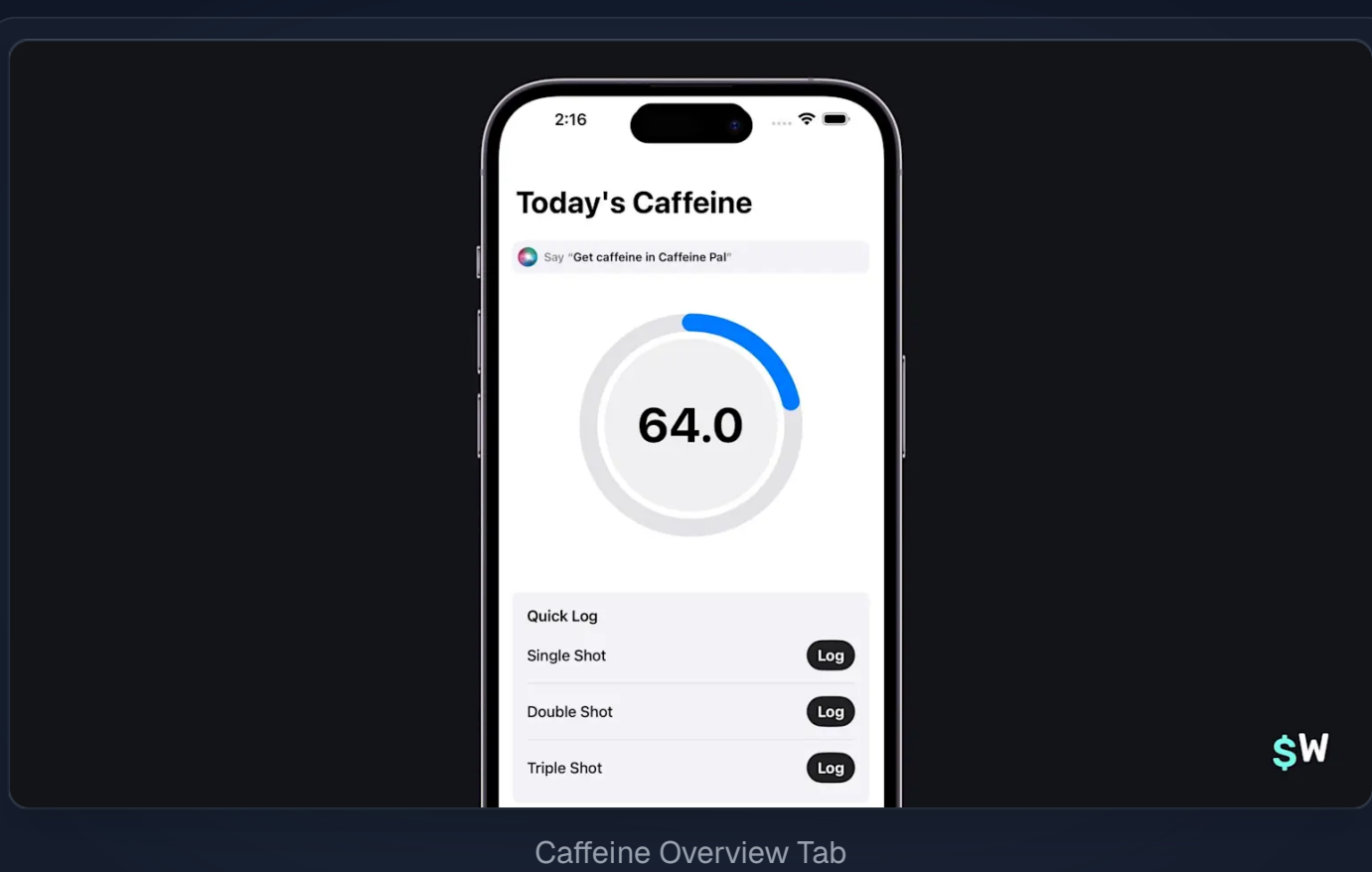
1. Provides a name for the intent.
2. A description of what it does.
3. And, it has a function to perform that action.

Using that information, let's make an intent for Caffeine Pal. We will go step-by-step, but here's the finished code for the project if you prefer to follow along that way.

A few things about the demo app. One, the data is fake — we don't actually use HealthKit to log caffeine. Secondly, you may remember this app from a few other tutorials we've used it for (StoreKit2 or StoreKit Paywall Views). For our purposes, I've just hardcoded in that a user has a "Pro" membership so we can focus on App Intents.

Our first App Intent

To begin, let's define what we want our intent to do exactly. In this screen, we see an overview of how much caffeine we've had today:



Caffeine Overview Tab

Showing an overview of how much caffeine we've had seems like a great candidate for an App Intent. Let's look at the pieces involved, as it stands now, to show that caffeine intake within the app. This will give us an idea of what pieces we need to do the same thing in an intent:

```
struct IntakeView: View {
    @Environment(PurchaseOperations.self) private var storefront: Purc
    @Environment(CaffeineStore.self) private var store: CaffeineStore

    var body: some View {
        NavigationStack {
            ScrollView {
                CaffeineGaugeView()
                    .padding(.bottom, 32)
                    .padding(.top)
                if store.amountOver > 0 {
                    AmountOverBannerView()
                        .padding()
                        .transition(.scale(scale: 0.8, anchor: .center
                }
                QuickLogView()
                    .padding()
            }
            .animation(.spring, value: store.amountOver)
            .navigationTitle("Today's Caffeine")
        }
    }
}
```

swift Copy

What sticks out to me is the `CaffeineStore` class. We'll likely want to reuse that for an intent dealing with caffeine. This is important because it brings us to the first lessons in making App Intents:

1. You need important classes like this to likely be a part of several targets (for widgets, live activities, etc). The same goes for "router" utility classes for navigation, and similar core functionality you'd like to use in a widget or intent.
2. It's a dependency, so we'll want a way to access it easily — whether that's via a Singleton or some other means.

If you think about those two things upfront, you'll be well-equipped to make all sorts of App Intents. Let's add a new file, and call it `GetCaffeineIntent.swift`:

```
struct GetCaffeineIntent: AppIntent {
    static var title = LocalizedStringResource("Get Caffeine Intake")
    static var description = IntentDescription("Shows how much caffein

    func perform() async throws -> some IntentResult {
        let store = CaffeineStore.shared
        let amount = store.amountIngested
        return amount
    }
}
```

swift Copy

This has all of the three things we mentioned above:

1. It has a title ("Get Caffeine Intake").
2. A description of what happens when we use it ("Shows much much caffeine you've had today.")
3. And, an implementation of that action, vended via the `perform` function.

However, if we build and run — we'll get a compiler error:

```
Return type of instance method 'perform ()' requires that 'Double' con
```

swift Copy

Looking at the return type, it's `some IntentResult`. **This is critical to understand to avoid a lot of undue frustration with App Intents.** You always return *some* form of an `IntentResult`. For example, if your intent just does an action, and has nothing of value to say about that action — you can simply return `.result()`. You don't *ever* return some primitive or domain specific type like we've done above.

Ours, though? It would be useful to tell the user how much caffeine they've had **and** return the actual amount, so change the return type to mark the intent to return *two* things:

1. An actual `Double` value of how much caffeine has been consumed.
2. And, some dialog to speak out their caffeine for the day.

So, instead of `some IntentResult`, here's what we need:

```
func perform() async throws -> some IntentResult & ReturnsValue<Double
    let store = CaffeineStore.shared
    let amount = store.amountIngested
    return .result(value: amount,
                   dialog: .init("You've had \(store.formattedAmount(f
}
```

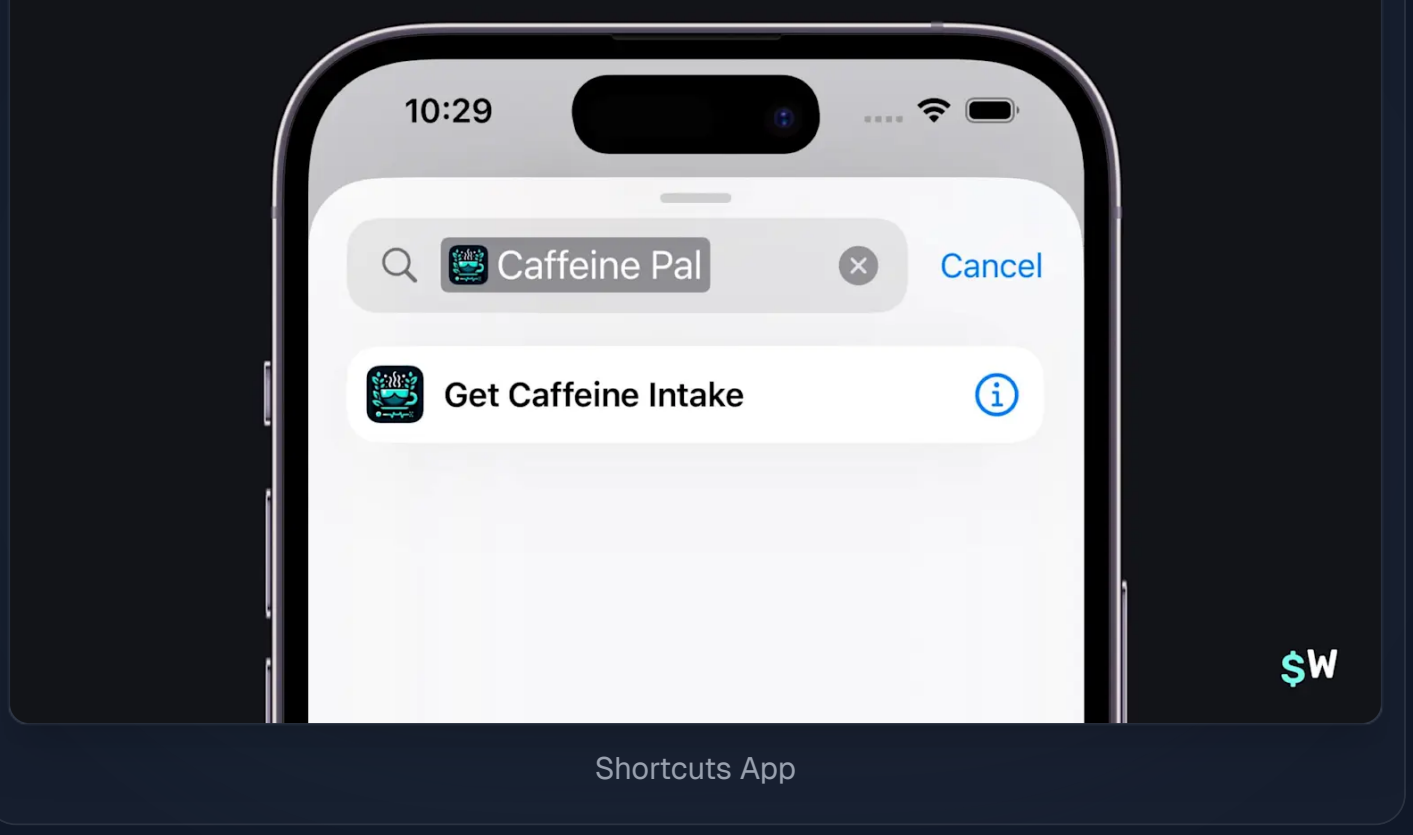
swift Copy

Each intent's return type needs to start with `some Intent` opaque return type, but from there we can also include more specific types. Here, we've noted that we return a double value and speak out dialog.

And that's *it*. We have a fully functional App Intent!

App Shortcuts and Siri

With that one intent definition, we get a lot of mileage in the system. If you open up the Shortcuts app, you'll see it listed there:



Shortcuts App

Now, users can string together all sorts of custom shortcuts using it. And, since we've returned a `Double` from the intent, it's flexible for several use cases. Perhaps someone wants to chart what their intake was over a week, or make a health dashboard using shortcuts. All are possible using our intent.

Next, it's also available for Siri to use. Just say "Get caffeine intake in Caffeine Pal", and the intent will run and speak out the answer. That's *a lot* of functionality for so little code.

Further, you can increase visibility by using some of these APIs:

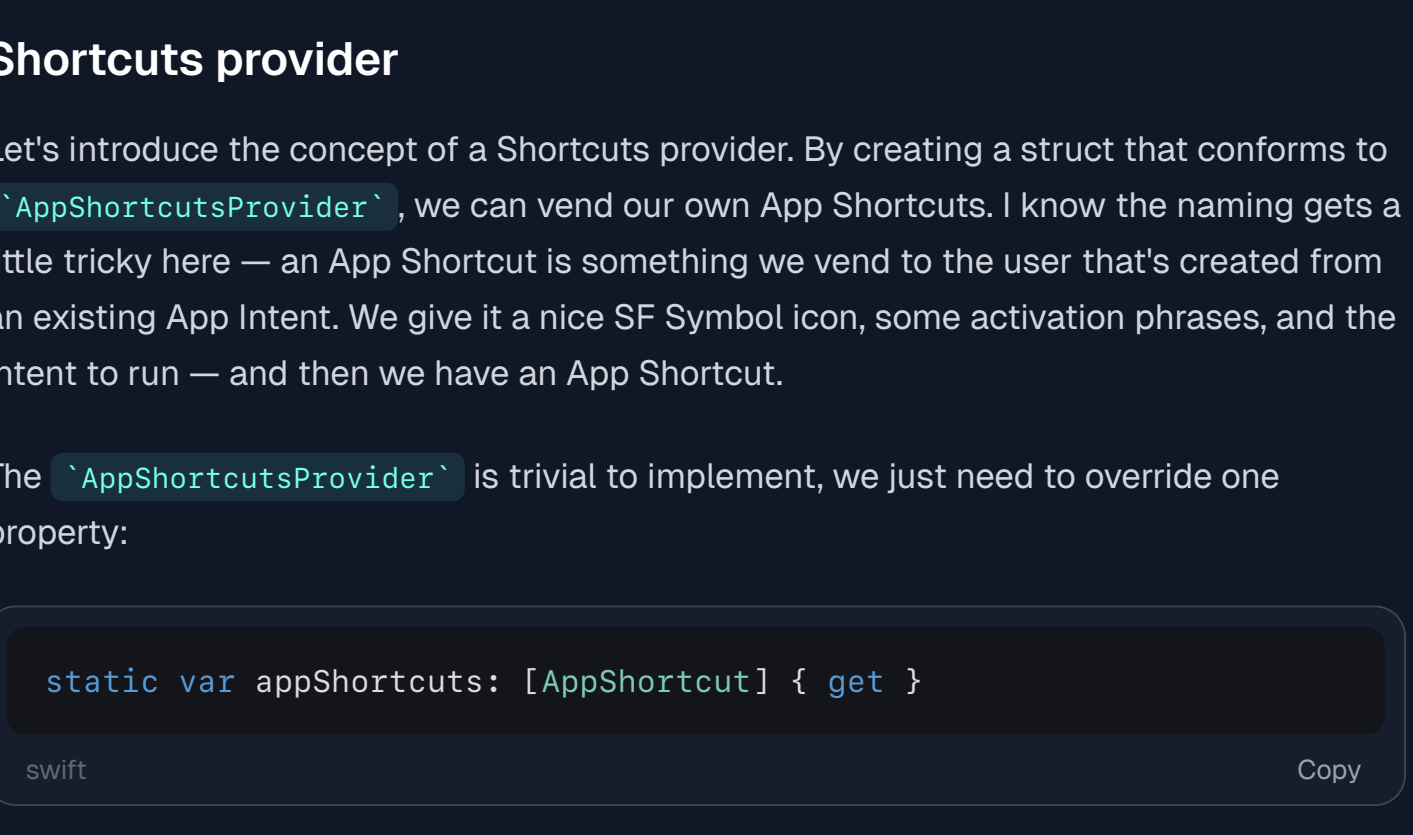
- A ShortcutsLink is a button that will deep link directly into the Shortcuts App, displaying all of your app's available actions.
- If a user says "Hey Siri, what can I do here?", Siri will list out your intents.
- A `SiriTipView`, which prompts the user to activate an intent from a phrase.

Let's give the `SiriTipView` a try. In `IntakeView`, I'll add one at the top of the `body`:

```
var body: some View {
    NavigationStack {
        ScrollView {
            // Show Siri tip
            SiriTipView(intent: GetCaffeineIntent())
                .padding()
            CaffeineGaugeView()
                .padding(.bottom, 32)
                .padding(.top)
            if store.amountOver > 0 {
                AmountOverBannerView()
                    .padding()
                    .transition(.scale(scale: 0.8, anchor: .center))
            }
            QuickLogView()
                .padding()
        }
        .animation(.spring, value: store.amountOver)
        .navigationTitle("Today's Caffeine")
    }
}
```

swift Copy

With that, users will see a call-to-action to try out the intent I passed into the tip view. Once they dismiss it, it won't show again:



Siri Tip View Suggesting App Intents

Let's take things even further, without even having to change our intent code, and bring it over to Spotlight search.

Shortcuts provider

Let's introduce the concept of a Shortcuts provider. By creating a struct that conforms to `AppShortcutsProvider`, we can vend our own App Shortcuts. I know the naming gets a little tricky here — an App Shortcut is something we vend to the user that's created from an existing App Intent. We give it a nice SF Symbol icon, some activation phrases, and the intent to run — and then we have an App Shortcut.

The `AppShortcutsProvider` is trivial to implement, we just need to override one property:

```
static var appShortcuts: [AppShortcut] { get }
```

swift Copy

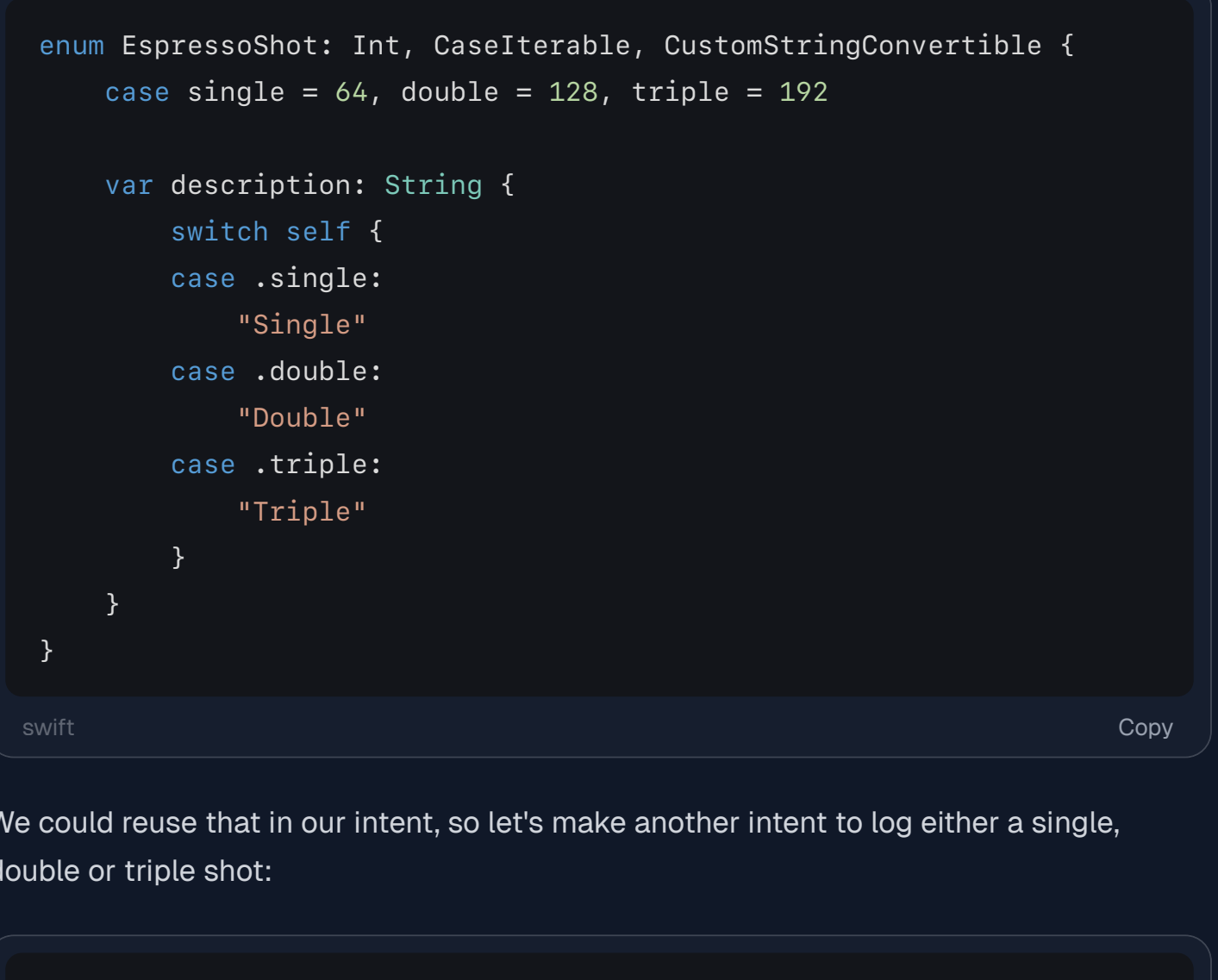
Here's our implementation:

```
struct ShortcutsProvider: AppShortcutsProvider {
    static var appShortcuts: [AppShortcut] {
        AppShortcut(intent: GetCaffeineIntent(),
                    phrases: ["Get caffeine in \(.applicationName)",
                              "See caffeine in \(.applicationName)",
                              "Show me much caffeine I've had in \(.ap
                              "Show my caffeine intake in \(.applicati
                    shortTitle: "Get Caffeine Intake",
                    systemImageName: "cup.and.saucer.fill")
    }
}
```

swift Copy

When you supply an struct that adopts `AppShortcutsProvider` — you don't even have to do anything else! The App Intents framework automatically picks up a provider adopter, and then it handles registration for you. That means your App Shortcuts are ready to use and discover, even if the app hasn't been opened yet.

Now, we can see our intent in Spotlight search, and it's ready to use inline with our app icon, too. Or, we can even search for the intent directly — and it'll show up in to activate:



Spotlight Search with App Intents

Another fun thing? If someone opens your app and says, "Hey Siri, what can I do here?" — Siri will bring up your App Shortcuts to try out.

Intents with parameters

A powerful part of App Intents is the ability to supply parameters. This means that if we wanted to make an intent that allowed for the user to log a shot of espresso, we could make a parameter for it.

We already have an `Enum` for espresso shots:

```
enum EspressoShot: Int, CaseIterable, CustomStringConvertible {
    case single = 64, double = 128, triple = 192

    var description: String {
        switch self {
        case .single:
            "Single"
        case .double:
            "Double"
        case .triple:
            "Triple"
        }
    }
}
```

swift Copy

We could reuse that in our intent, so let's make another intent to log either a single, double or triple shot:

```
struct LogEspressoIntent: AppIntent {
    static var title = LocalizedStringResource("Log Espresso Shot")
    static var description = IntentDescription("Logs some espresso.")

    @Parameter(title: "Shots")
    var shots: EspressoShot?

    static var parameterSummary: some ParameterSummary {
        Summary("Logs \(\.$shots) of caffeine")
    }

    init() {}

    init(shots: EspressoShot) {
        self.shots = shots
    }

    func perform() async throws -> some IntentResult & ProvidesDialog
        if shots == nil {
            shots = try await $shots.requestValue(.init(stringLiteral:
        }

        let store: CaffeineStore = .shared
        store.log(espressoShot: shots!)

        // Refresh widgets
        WidgetCenter.shared.reloadAllTimelines()

        return .result(dialog: .init("Logged \(store.formattedAmount(.
    }
}
```

swift Copy

It has the same makeup of our previous intent, only now — we have use a property wrapper to specify a parameter we require:

```
@Parameter(title: "Shots")
 var shots: EspressoShot?
```

swift Copy

That on its own wouldn't compile though. Why? Because if you reuse your app's models or enums, there are special types you make for them to be exposed to intents: `AppEntity` for models, and `AppEnum` for enumerations. Right below it, we could conform to `AppEnum` so that we can use it as a parameter in our intent:

```
extension EspressoShot: AppEnum {
    static var typeDisplayRepresentation: TypeDisplayRepresentation =
    static var typeDisplayName: LocalizedStringResource = "Shots"
    static var caseDisplayRepresentations: [EspressoShot: DisplayRepre
        .single: "Single",
        .double: "Double",
        .triple: "Triple"
    ]
}
```

swift Copy

A few other important things I want to point that will save you a lot of debugging heartache later:

1. When you have an intent with parameters, you need to supply a default initializer. It's a good idea to supply another one which takes in your parameters too. We've done that in our intent — there's an empty `init`, and a `init(shots: EspressoShot)`. If you miss this, you'll hit some weird edge cases where your

intent won't fire without any obvious cause (when in fact, the cause is the missing default initializer).

2. You can have the framework prompt the user for a value, you'll see that in our `perform` function where if there is no espresso shot supplied, we prompt for one using `$shots.requestValue()`.

With that, let's add it to our `AppShortcutsProvider` so it'll start showing up in Spotlight and Siri:

```swift
struct ShortcutsProvider: AppShortcutsProvider {
    static var shortcutTileColor: ShortcutTileColor {
        return .lightBlue
    }

    static var appShortcuts: [AppShortcut] {
        AppShortcut(intent: GetCaffeineIntent(),
                    phrases: ["Get caffeine in \(.applicationName)",
                              "See caffeine in \(.applicationName)",
                              "Show me much caffeine I've had in \(.ap
                              "Show my caffeine intake in \(.applicati
                    shortTitle: "Get Caffeine Intake",
                    systemImageName: "cup.and.saucer.fill")
        AppShortcut(intent: LogEspressoIntent(),
                    phrases: ["Log caffeine in \(.applicationName)",
                              "Log espresso shots in \(.applicationNam
                    shortTitle: "Get Caffeine Intake",
                    systemImageName: "cup.and.saucer.fill")
    }
}
```

swift Copy

And just like that, we've got an intent that takes in a parameter.

Interactive widgets

With our two intents all done, we can reuse them in other places too. In **WidgetKit**, we could create an interactive widget that logs some caffeine. I'm not going to go into the specifics of how widgets work (Apple has a great write up over **here**) — but just know that starting in iOS 17, a button or toggle can use an intent in an initializer.

If we open up `CaffeinePalEspressoWidget`, in the `Button` — we simply initialize our `LogEspressoIntent` with a single shot:

```swift
struct LogEspressoWidgetView : View {
    let store: CaffeineStore = .shared
    var entry: EspressoTimelineProvider.Entry

    var body: some View {
        VStack(alignment: .leading) {
            Text("Today's Caffeine:")
                .font(.caption)
                .padding(.bottom, 4)
            Text(store.formattedAmount(.init(value: entry.amount, unit
                .font(.caption.weight(.semibold))
                .foregroundStyle(Color.secondary)
            Spacer()
            // Our intent being reused
            Button(intent: LogEspressoIntent(shots: .single)) {
                Text("Log a Shot")
                    .frame(minWidth: 0, maxWidth: .infinity)
            }
        }
    }
}
```

swift Copy

Again, we're already seeing the flexibility of the parameters and initializers we made for our `LogEspressoIntent` coming into play. By passing in a single shot, it makes for a perfect interactive widget to quickly log shots from your Home Screen:

Interactive Widget using App Intents

This is a good time to talk about intent reuse, too. Since we don't want to duplicate the logic to add caffeine and espresso, we should simply call the intent within the main app target too. So these buttons to log espresso within the main app...

Logging Shots in the Intake View

...can be written just like the WidgetKit extension, meaning all of the caffeine logic is now housed into one place. We'll have those buttons use the `LogEspressoIntent` too:

```swift
struct QuickAddButton: View {
    @Environment(PurchaseOperations.self) private var storefront: Purc
    @Environment(CaffeineStore.self) private var store: CaffeineStore

    let text: String
    let shots: EspressoShot

    var body: some View {
        HStack {
            Text(text)
                .fontWeight(.medium)
            Spacer()
            // Our LogEspressoIntent in use once again
            Button(intent: LogEspressoIntent(shots: shots)) {
                Text("Log")
                    .foregroundStyle(Color.inverseLabel)
                    .fontWeight(.bold)
            }
            .buttonBorderShape(.capsule)
            .buttonStyle(.borderedProminent)
        }
        .padding(.vertical, 6)
    }
}
```

swift Copy

Controls

It doesn't stop there, though. We could even use the same intents to power the **Control APIs**, announced with iOS 18. We could add a control widget to show on the Lock Screen and in Control Center to show how much caffeine we've had for the day.

It looks nearly identical to how we created the interactive widget, only we use a `ControlWidget` this time:

```swift
struct CaffeinePalWidgetsControl: ControlWidget {
    static let kind: String = "com.superwall.caffeinePal.Caffeine-Pal.
    var body: some ControlWidgetConfiguration {
        StaticControlConfiguration(kind: Self.kind) {
            // Use our App Intent once again
            ControlWidgetButton(action: GetCaffeineIntent()) {
                Label("Caffeine Intake", systemImage: "cup.and.saucer.
            }
        }
        .displayName("Get Caffeine Intake")
        .description("Shows how much caffeine you've had today.")
    }
}
```

swift Copy

Again, you're seeing a special button type, `ControlWidgetButton`, use our existing `GetCaffeineIntent`. And with that little code, it's ready to go:

Lock Screen and Control Center Widgets

Wrapping up

App Intents take your app's best parts, and it spreads them all throughout the system. It's a can't-miss API. Here, without much more than a 100 lines of code, we've made two App Intents. Using them, we:

- Supported Siri
- The Shortcuts App
- Spotlight Search
- Interactive Widgets
- Control widgets

And, that's not even all you can do. Once Apple Intelligence is ready to go, we can hook into the system in more powerful ways by adopting `Transferable` and specific schemas to make our intents more flexible. While we can't really test that today, this **session** from W.W.D.C. 2024 gives us a glimpse of how we can do it.

Remember, making an App Intent is a simple as adopting `AppIntent`, and implementing a `perform` function. It used to be *a lot* more work than that! As long as you can access objects from your code base for interacting with data, models and an API — you can go crazy with making as many intents as you can think of.

Of course, if you're looking for a way to make paywalls as flexible and powerful as your App Intents — look no further than Superwall. Sign up for a free account today to get started **testing paywalls in minutes**.

Get in touch

Get personalized support

What you'll get:

- ✓ Personalized demo of Superwall's paywall builder
- ✓ Expert guidance on monetization strategy
- ✓ Custom implementation recommendations
- ✓ Access to 200+ proven paywall templates
- ✓ Revenue optimization best practices

Quick & Easy

Most meetings take 30 minutes or less. We'll show you exactly how Superwall can increase your app revenue.

Contact Form

First Name
Last Name
Email
Company
Monthly Revenue: Select an option...
Meeting Type: Select an option...

Submit ›

© 2025 Superwall. All rights reserved.